

Security Group Exceptions – Michael Waterman

 michaelwaterman.nl/2024/06/02/mastering-applocker-security-group-exceptions

Michael Waterman

June 2, 2024



I'll promise to keep this blog post short—well, shorter than usual (hopefully). Last week, I worked on a project involving application allow listing. In the Windows ecosystem, this can be achieved using Windows AppLocker. While AppLocker has been around for quite some time, it's only recently become available on Windows 11 Professional. Previously, it was an exclusive feature for Enterprise versions.

You might think Microsoft finally came to its senses by allowing this functionality on non-Enterprise editions, but the reality is different. There's an unresolved bug that downgrades Enterprise versions of Windows to Professional when they are cloud-connected. This downgrade causes AppLocker policies to be ignored, which, as you can imagine, is a security incident in itself. Instead of fixing the bug, Microsoft decided to allow AppLocker on Windows 11 Professional as well. This change made me happy because AppLocker is really needed. Let me explain why that is.

What is AppLocker?

In short, AppLocker is a security feature integrated into the Windows operating system that enables administrators to control which applications and executable files users can run. This feature has been a cornerstone in managing application execution within enterprises, providing a robust layer of security by preventing unauthorized or potentially harmful software from running on the network.

AppLocker goals

The primary goals of AppLocker are to enhance security, control software usage, and simplify IT management within an organization. By restricting which applications can run, AppLocker significantly reduces the risk of malware and unauthorized software execution, thereby protecting sensitive data and maintaining the integrity of the operating environment. Additionally, AppLocker enforces policies that ensure only approved applications are used, aiding in compliance with corporate policies and industry regulations. Furthermore, by providing a centralized way to manage application policies, AppLocker simplifies the administrative workload, enabling IT departments to efficiently enforce and update rules, thus maintaining a secure and compliant IT infrastructure.

AppLocker applicability

AppLocker is a versatile and powerful tool for managing application execution policies across various Windows environments. Its applicability spans several versions of Windows, each offering different levels of support and features. Understanding where you can use AppLocker is crucial for effective deployment and management.

Supported Windows Versions

1. Windows Server:

AppLocker is fully supported on Windows Server editions, starting from Windows Server 2008 R2 and onward. This includes Windows Server all current supported versions.

2. Windows Enterprise:

Historically, AppLocker has been an exclusive feature for Windows Enterprise editions, including Windows 7 Enterprise, Windows 8/8.1 Enterprise, Windows 10 Enterprise, and Windows 11 Enterprise. These editions offer full functionality for creating and managing AppLocker policies.

3. Windows Professional:

Within the lifetime of Windows 11, AppLocker is now also available on Windows 11 Professional. This marks a significant expansion from its previous exclusivity to Enterprise editions, allowing more businesses and individual users to benefit from its robust application control capabilities.

AppLocker is particularly effective in corporate networks where stringent control over application execution is necessary or desired to maintain security and compliance. It helps in preventing the execution of unauthorized software and reduces the risk of malware infections. Moreover, in every company asset management and application portfolio control are critical for ensuring that only approved and necessary software is used, helping to manage licensing costs and reduce unnecessary software bloat.

What does AppLocker prevent?

AppLocker is designed to prevent a variety of security risks that could compromise an organization's IT environment. It restricts the execution of unapproved software, ensuring that only trusted applications are used within the organization. By blocking unknown or untrusted applications, AppLocker mitigates the risk of malware and ransomware attacks that could jeopardize system security and data integrity. It also prevents unintended software installations by users, which could lead to system instability or introduce security vulnerabilities. Additionally, by controlling which applications can run, AppLocker reduces the risk of insider threats, where employees might deliberately or inadvertently run harmful software. Overall, AppLocker acts as a safeguard against various forms of software-based threats, enhancing the overall security posture of the organization.

How does AppLocker work?

Understanding how AppLocker works is crucial to leveraging its full potential in securing your organization's IT environment. AppLocker operates by defining and enforcing rules that govern the execution of applications, dlls (which are executable as well), MSI installers, scripts, and Windows store apps. These rules are based on various attributes of the files, allowing for granular control over what can and cannot run on your network. By using these rules, administrators can create a controlled and secure environment where only authorized software is permitted to execute.

AppLocker employs several methods to identify and control applications:

1. **Publisher:** This method uses the digital signature of the software publisher to identify applications. Administrators can create rules that allow or block applications from specific vendors, ensuring that only software from trusted sources is executed.
2. **Path:** AppLocker can control application execution based on the file location. This is particularly useful for allowing or blocking software stored in specific directories. For example, administrators can permit applications only from a secured folder that contains verified software.
3. **File Hash:** This method uses a cryptographic hash to uniquely identify files. By creating rules based on file hashes, administrators can ensure that only specific versions of an application are allowed to run. This is an effective way to prevent tampered or altered versions of software from executing.

These identification methods allow AppLocker to apply rules with precision, targeting specific applications while leaving others unaffected. Administrators can assign these rules to particular users or security groups, providing tailored access controls that match the needs of different departments or roles within the organization. By doing so, AppLocker ensures that users have access to the applications they need for their work while maintaining strict control over what software can run on the system.

Moreover, AppLocker integrates seamlessly with the Group Policy management infrastructure and Intune, allowing administrators to deploy and manage application control policies across the entire network from a central location. This centralized management capability simplifies the process of updating and maintaining AppLocker rules, ensuring that security policies remain consistent and up-to-date across the organization.

Overall, AppLocker's robust rule-based system and its integration with Windows security infrastructure make it a powerful tool for preventing unauthorized application execution, enhancing security, and simplifying IT management.

Understanding AppLocker allow & deny rule collections

Each AppLocker rule collection operates as a precise allow list of files, ensuring that only specified applications can run. Essentially, files can only be executed if they fall under one or more allow rules within the rule collection. Additionally, you can create rules that explicitly deny certain files from running, adding another layer of control. Any files not explicitly allowed or denied by these rules are implicitly blocked from execution. Grasping this "block by default, allow by exception" behavior is crucial for understanding the impact of your policy on users within your organization.

When AppLocker applies its rules, it follows a specific order of operations. First, it checks for any explicit deny actions within the rule list. If a file is denied from running by a rule, this deny action takes precedence over any allow actions and cannot be overridden (Remember this). Following this, AppLocker searches for any explicit allow actions for the file in question. Given that AppLocker functions as an allow list by default, if no rule explicitly allows or denies a file, the default deny action blocks it from running. This systematic approach ensures that only authorized applications are executed, thereby maintaining a secure and controlled IT environment.

Understanding this hierarchical rule application process is essential for administrators. It helps in designing effective policies that balance security with functionality, ensuring that legitimate applications are not inadvertently blocked while preventing unauthorized software from running. By mastering the intricacies of AppLocker rule collections, you can fine-tune your application control strategy to meet the specific needs of your organization, enhancing both security and operational efficiency.

Testing and deployment

In case you want to get fired, skip this paragraph, but I really would like you to keep your job, so please read the following:

Before deploying AppLocker policies in a production environment, it is crucial to thoroughly test them in a controlled setting. This step ensures that the policies do not inadvertently

disrupt legitimate applications, which could cause significant operational issues. One of the most effective ways to conduct this testing is by using AppLocker's Audit mode.

Audit mode allows administrators to monitor which applications would be blocked or allowed by the policies without actually enforcing the restrictions. By doing this, you can see the potential impact of the policies and make necessary adjustments before full implementation. This fine-tuning process is essential to create a balance between security and functionality, ensuring that critical applications remain operational while blocking unauthorized software.

Testing in a controlled environment, coupled with the insights gained from Audit mode, provides a comprehensive understanding of how AppLocker policies will behave once they are deployed. This approach minimizes the risk of disruption and helps in creating robust, effective application control policies that enhance security without compromising productivity.

In summary, always start with testing and use Audit mode to refine your policies. This careful preparation is key to successfully integrating AppLocker into your organization's security framework.

Getting started with AppLocker

Setting up AppLocker from for the first time involves a few steps to get everything configured properly. For the sake of completeness, and if you've never used AppLocker before, here's a chapter on how to get started. It's crucial to perform these steps on a test system, not a production system, to ensure everything works as expected before rolling it out organization-wide. Here's how you can get started:

Step 1: Open the Group Policy Editor

1. Press Win + R, type `gpedit.msc`, and press Enter to open the Group Policy Editor.

Step 2: Navigate to the AppLocker Policies

1. In the Group Policy Editor, navigate to **Computer Configuration > Windows Settings > Security Settings > Application Control Policies > AppLocker**.

Step 3: Configure Executable Rules

1. Right-click on **Executable Rules** and select **Create Default Rules**. This will create basic rules that allow all files in the Windows folder and Program Files folder to run. Repeat this for **Windows Installer rules**, **Script rules**, **Packaged app rules** and optionally **DLL Rules** if you have them activated.

Application Control Policies					
AppLocker					
Executable Rules					
Windows Installer Rules					
Script Rules					
DLL Rules					
Packaged app Rules					

Action	User	Name	Condition	Exceptions
Allow	Everyone	(Default Rule) All files located in the Pro...	Path	
Allow	BUILTIN\Administrators	(Default Rule) All files	Path	
Allow	Everyone	All files located in the Windows folder	Path	

Step 5: Enable the AppLocker Policy

1. Go back to the **AppLocker** node and right-click on it.
2. Select **Properties**.
3. In the **AppLocker Properties** dialog, check the boxes to enable the rule enforcement for **Executable rules**, **Windows Installer rules**, **Script rules**, and **Packaged app rules** as needed.
4. Click **OK** to apply the changes.

AppLocker Properties

Enforcement Advanced

Specify whether AppLocker rules are enforced for each rule collection.

Executable rules:

☒ Configured

Enforce rules

Windows Installer rules:

☒ Configured

Enforce rules

Script rules:

☒ Configured

Enforce rules

Packaged app Rules:

☒ Configured

Enforce rules

DLL rules:

☒ Configured

Enforce rules

[More about rule enforcement](#)

OK Cancel Apply

Step 6: Deploy the Policy

1. To ensure the AppLocker policy is applied across the organization, link the Group Policy Object (GPO) containing the AppLocker rules to the appropriate organizational units (OUs) in Active Directory.

By following these steps, you can set up AppLocker to start controlling which applications can run on your systems, providing an essential layer of security. Remember, starting with the default rules is a good practice to ensure critical system files and applications continue to function correctly while you refine your custom rules.

Problem statement

In my recent project, I aimed to create an AppLocker policy that denies all known “Living off the Land” binaries (Lolbins) based on publisher rules. Lolbins are legitimate system binaries often exploited in attacks. I compiled a list of these binaries from [LOLBas Project](#).

However, during the construction of this policy, I encountered a significant limitation: AppLocker’s rule precedence. When a file is denied for the “everyone” group, it cannot be made available to specific security groups that require its functionality. Let’s use “MMC.exe” as an example. This particular Lolbin can bypass User Account Control (UAC) if you already have admin credentials. While it poses a security risk in the hands of administrators, it is safe for regular users who need it for specific tasks. Unfortunately, an explicit deny rule for “everyone” means there’s no straightforward way to make exceptions for particular security groups.

Setting up an explicit deny rule

For completeness and to demonstrate the process, I’ll include the steps for creating an explicit deny rule for mmc.exe. Using a publisher rule is the preferred and most flexible way, as it allows you to manage applications based on the software’s digital signature rather than its specific path, making it easier to maintain and update policies.

Step 1: Open the Group Policy editor

1. Press Win + R, type gpedit.msc, and press Enter to open the Group Policy Editor.

Step 2: Navigate to AppLocker policies

1. In the Group Policy Editor, navigate to **Computer Configuration > Windows Settings > Security Settings > Application Control Policies > AppLocker > Executable Rules**.

Step 3: Create a New Rule

1. Right-click on **Executable Rules** and select **Create New Rule**.

Step 4: Start the Create New Rule Wizard

1. The **Create New Rule** wizard will open. Click **Next** on the introduction screen.

Step 5: Select the Rule Action

1. Choose **Deny** and click **Next**.

Step 6: Specify the User or Group

1. Select **Everyone** to apply this rule to all users, and then click **Next**.

Step 7: Choose the Condition Type

1. Select **Publisher** as the condition type and click **Next**.

Step 8: Configure the Publisher Rule

1. Click on **Browse** and navigate to `C:\Windows\System32\mmc.exe` to select the file. This allows AppLocker to extract the digital signature information.
2. After selecting the file, adjust the slider to the desired publisher level (Publisher, Product name, File name, or File version). For flexibility, you might want to set it at the File name level.
3. Click **Next** and **Next** again at the Exceptions page.

The screenshot shows the 'Create Executable Rules' wizard in the 'Publisher' step. The left sidebar has a navigation pane with 'Before You Begin', 'Permissions', 'Conditions', 'Publisher' (selected), 'Exceptions', and 'Name'. The main area contains instructions: 'Browse for a signed file to use as a reference for the rule. Use the slider to select which properties define the rule; as you move down, the rule becomes more specific. When the slider is in the any publisher position, the rule is applied to all signed files.' Below this, the 'Reference file' is set to 'C:\Windows\System32\mmc.exe'. A slider is positioned over the 'File name' property, which is highlighted with a red box. The 'File name' field contains 'MMC.EXE'. Other properties shown include 'Publisher' (O=MICROSOFT CORPORATION, L=REDMOND, S=V) and 'Product name' (MICROSOFT® WINDOWS® OPERATING SYSTEM). At the bottom, there are buttons for '< Previous', 'Next >', 'Create', and 'Cancel'.

Step 9: Create the Rule

1. Review the rule settings to ensure everything is correct.

2. Click **Create** to finalize the rule.

Step 10: Confirm the Rule

1. The new rule should now appear in the list of executable rules. Verify that it's listed as a **Deny** rule for **mmc.exe** under **Executable Rules**.

Action	User	Name	Condition	Exceptions
 Allow	Everyone	(Default Rule) All files located in the Pro...	Path	
 Allow	BUILTIN\Administrators	(Default Rule) All files	Path	
 Allow	Everyone	All files located in the Windows folder	Path	
 Deny	Everyone	MMC.EXE, in MICROSOFT® WINDOWS...	Publisher	

Step 11: User Experience

When a user attempts to execute mmc.exe, they will encounter a notification indicating that the action has been blocked by system policy. The exact message typically states something like, “This app has been blocked by your system administrator,” ensuring that the user understands the restriction is intentional and managed by IT policy. This pop-up reinforces the security measures in place and guides users to contact their IT department for further assistance if needed.

Exception management

AppLocker is fundamentally an allow list feature, but it also offers a powerful mechanism to create exceptions within its rules. For instance, consider the default rule “*All files located in the Windows folder*,” which allows any executable within the Windows directory to run. In the exception tab of this rule, you can add an application like mmc.exe, effectively telling AppLocker to block mmc.exe. Instead of using an explicit deny, you can create a more specific rule that allows (or denies) the application to run for designated security groups later on.

Setting up an exception rule

To ensure certain applications are blocked while allowing most others in the Windows folder to run, you can add an exception by publisher rule to the default “All files located in the Windows folder” rule. Here’s how to do it:

Please note! remove the previously created deny rule first.

Step 1: Open the Group Policy editor

1. Press Win + R, type gpedit.msc, and press Enter to open the Group Policy Editor.

Step 2: Navigate to AppLocker Policies

1. In the Group Policy Editor, navigate to **Computer Configuration > Windows Settings > Security Settings > Application Control Policies > AppLocker > Executable Rules**.

Step 3: Edit the Default Rule

1. Locate the default rule **All files located in the Windows folder**.
2. Right-click on this rule and select **Properties**.

Step 4: Add an exception

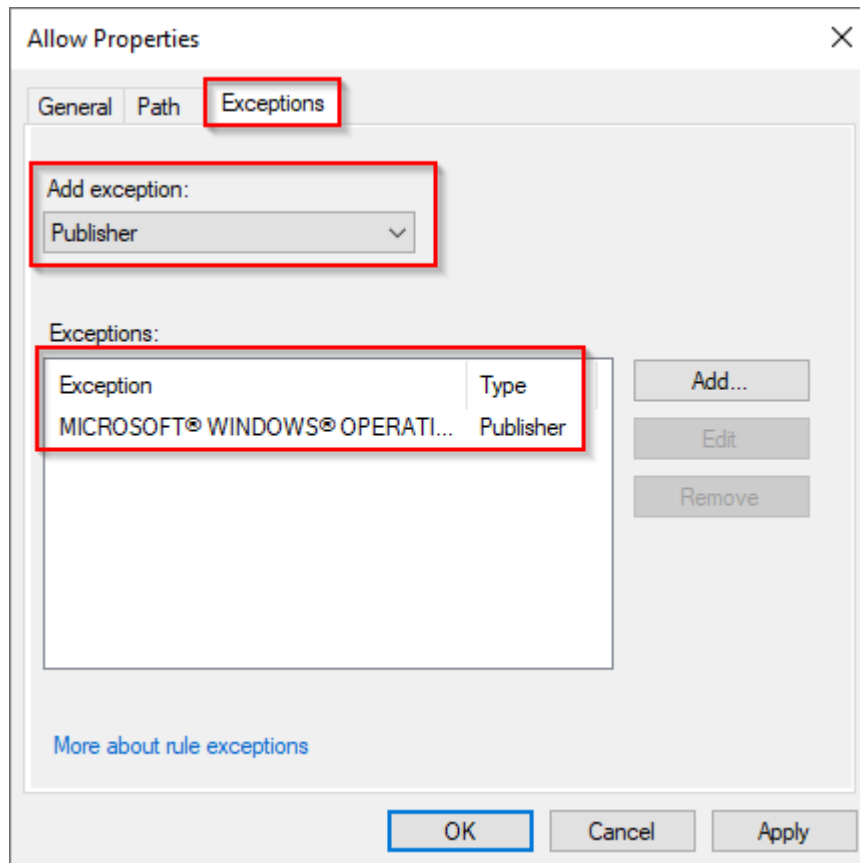
1. In the **Properties** window, go to the **Exceptions** tab.
2. Click **Add** to create a new exception.

Step 5: Choose the Condition type

1. Select **Publisher** as the condition type for the exception and click **Next**.

Step 6: Configure the Publisher Rule

1. Click on **Browse** and navigate to the file you want to exclude, for example, `C:\Windows\System32\mmc.exe`. This allows AppLocker to extract the digital signature information.
2. After selecting the file, adjust the slider to the desired publisher level (Publisher, Product name, File name, or File version). For flexibility, you might want to set it at the File name level.
3. Click **Next**.



Step 7: Finalize the exception

1. Review the exception settings to ensure everything is correct.
2. Click **Create** to finalize the exception.

Step 8: Confirm the exception

1. Ensure that the exception is listed in the **Exceptions** tab under the default rule **All files located in the Windows folder**.
2. Click **OK** to save the changes.

Step 9: Apply and refresh policies

1. Close the Group Policy Editor.
2. Open Command Prompt as an administrator and run `gpupdate /force` to apply the new policy settings.

To clarify, by adding `mmc.exe` as an exception under a general allow rule, we are specifying that `mmc.exe` should not run. Then, we create a separate, specific rule that allows `mmc.exe` to execute, but only for the users within a particular security group. This nuanced approach lets us balance security with necessary functionality, ensuring that only the right users have access to potentially risky applications.

Let's set that up!

Implementing the full solution

Now that we have set up the default rules and explicitly denied mmc.exe from running by means of an exception, we can create a new rule that allows mmc.exe to run for a specific security group. Let's name this security group "SG_AppLocker_Allow_MMC"

Here's how you can implement this solution:

Step 1: Create the security group

1. Open **Active Directory Users and Computers**.
2. Navigate to the organizational unit (OU) where you want to create the group.
3. Right-click the OU, select **New > Group**.
4. Name the group "SG_AppLocker_Allow_MMC" and click **OK**.

Step 2: Create an exception allow rule

1. Right-click on **Executable Rules** and select **Create New Rule**.
2. In the **Create New Rule** wizard, select **Allow** and click **Next**.
3. Select **User or Group**, click **Select**, and add the "SG_AppLocker_Allow_MMC" group. Click **OK** and then **Next**.
4. Choose **Publisher** as the condition type.
5. Click on **Browse** and navigate to C:\Windows\System32\mmc.exe to select the file.
This allows AppLocker to extract the digital signature information.
6. Adjust the slider to the desired publisher level (Publisher, Product name, File name, or File version). For flexibility, you might want to set it at the File name level.
7. Click **Next**, review the settings, and then click **Create**.

Create Executable Rules

Conditions

Before You Begin

Permissions

Conditions

Publisher

Exceptions

Name

Select the type of primary condition that you would like to create.

☒ Publisher

Select this option if the application you want to create the rule for is signed by the software publisher.

☐ Path

Create a rule for a specific file or folder path. If you select a folder, all files in the folder will be affected by the rule.

☐ File hash

Select this option if you want to create a rule for an application that is not signed.

[More about rule conditions](#)

< Previous

Next >

Create

Cancel

Create Executable Rules

Publisher

Before You Begin

Permissions

Conditions

Publisher

Exceptions

Name

Browse for a signed file to use as a reference for the rule. Use the slider to select which properties define the rule; as you move down, the rule becomes more specific. When the slider is in the any publisher position, the rule is applied to all signed files.

Reference file:

C:\Windows\System32\mmc.exe

Browse...

- Any publisher

- Publisher:

O=MICROSOFT CORPORATION, L=REDMOND, S=V

- Product name:

MICROSOFT® WINDOWS® OPERATING SYSTEM

- File name:

MMC.EXE

- File version:

* And above

☐ Use custom values

< Previous

Next >

Create

Cancel

13/15

Create Executable Rules

Exceptions

Before You Begin
Permissions
Conditions
Publisher
Exceptions
Name

To add an exception, select the type of exception and then click Add. Exceptions are optional and allow you to exclude files that would normally be included in the rule. To continue configuring this rule without adding an exception, click Next.

Primary condition:
MMC.EXE, in MICROSOFT® WINDOWS® OPERATING SYSTEM, from O=MICROSOFT CORPORATION, L=REDMOND, S=WASHINGTON, C=US

Add exception:
Publisher

Exceptions:

Exception	Type
-----------	------

Add...
Edit
Remove

< Previous Next > Create Cancel

Create Executable Rules

Name and Description

Before You Begin
Permissions
Conditions
Publisher
Exceptions
Name

Enter a name to identify this rule.

Name:
Allow MMC for SG_AppLocker_Allow_MMC

Description: (Optional)
|

< Previous Next > Create Cancel

Step 3: Test the Policy

1. Log in as a member of the "SG_AppLocker_Allow_MMC" group and verify that mmc.exe runs without issues.
2. Log in as a regular user and confirm that mmc.exe is blocked.

Note! Verify that your test user is part of the security group with the command `whoami /groups`.

```
C:\>whoami /groups

GROUP INFORMATION
-----

Group Name                                     Type
ibutes
=====
Everyone                                     Well-known group
atory group, Enabled by default, Enabled group
BUILTIN\Users                               Alias
atory group, Enabled by default, Enabled group
NT AUTHORITY\INTERACTIVE                     Well-known group
atory group, Enabled by default, Enabled group
CONSOLE LOGON                               Well-known group
atory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users              Well-known group
atory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization                Well-known group
atory group, Enabled by default, Enabled group
LOCAL                                         Well-known group
atory group, Enabled by default, Enabled group
security\SG_AppLocker-Allow_MMC              Group
atory group, Enabled by default, Enabled group
Authentication authority asserted identity Well-known group
atory group, Enabled by default, Enabled group
Mandatory Label\Medium Mandatory Level      Label
```

By following these steps, you have successfully created a policy that denies mmc.exe from running for everyone by default while allowing it for members of the security group. This approach maintains security by blocking potential threats while providing necessary functionality to specific user groups.

Conclusion

By setting up AppLocker, you can enhance your security posture by allowing only trusted applications to run while maintaining flexibility for specific user groups. Denying risky binaries like mmc.exe by default, yet permitting them for the a specific security group, exemplifies a balanced approach to security and functionality. This step-by-step guide ensures you can implement similar policies in your organization.

Thank you for reading! If you have any feedback or questions, please feel free to leave a comment or contact me directly.